Cache Controller Design Assignment

Name: Muhammad Talha Javed
Course: Computer Architecture
Instructor: Sir Muhammad Yousaf

1. Address Format

The cache contains **8 cache lines**, while the main memory contains **32 memory lines**. Each cache line stores **4 words**, and each word contains **4 bytes**.

Therefore, the address is divided as follows:

Address Bits	Field	Description
31–9	Unused	Higher unused address bits
8–7	Tag	Identifies memory block
6–4	Index	Selects one of 8 cache lines
3–2	Word Offset	Selects one of 4 words
1–0	Byte Offset	Selects one byte inside a word

Calculation

Number of cache lines = 8 which is 2^3

Index bits = **3**

Each cache line stores
4 words
$4 = 2^2$

Word Offset = 2 bits
Each word contains 4 bytes which $4 = 2^2$ which leads to Byte Offset = **2 bits**

Main memory contains
32 blocks
$32 = 2^5$
so Total block address bits = **5**
Therefore
Tag = 5 − 3 = 2 bits

2. Number of Tag Bits

Main memory contains **32 blocks**, requiring **5 bits** to identify each block.

The cache contains **8 cache lines**, requiring **3 index bits**.

Therefore,

Tag Bits = 5 − 3

Tag Bits = 2

Hence, **2 tag bits** are required.

3. Number of Tag Registers

Each cache line stores one tag.

Cache lines = **8**

Therefore,

Number of Tag Registers = 8

Each register stores **2 bits**.

4. Number of Comparators

A Direct Mapped Cache compares only the selected cache line.

Therefore,

Number of Comparators = 1

Only one comparator is required because only one tag comparison is performed for each memory access.

5. Cache Controller Datapath

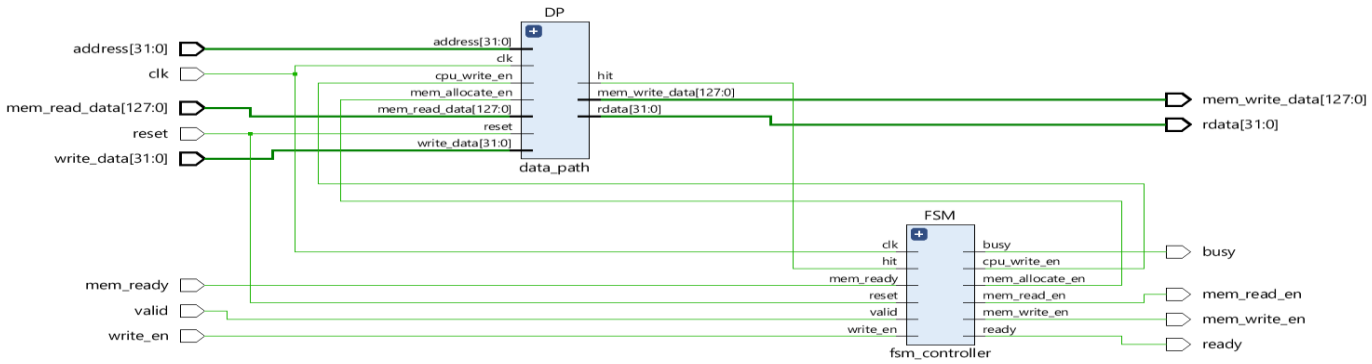
The datapath consists of the following components:

- CPU Address Input
- Tag Extraction Logic
- Index Extraction Logic
- Offset Extraction Logic
- Cache Data Array (8 × 128 bits)
- Tag Array (8 × 2 bits)
- Valid Bit Array
- Tag Comparator
- Word Selection Multiplexer
- Main Memory Interface

The datapath extracts the tag, index and offset from the address. The index selects the required cache line. The comparator compares the stored tag with the incoming tag. If both tags match and the valid bit is set, a cache hit occurs and the requested word is selected using the word offset. Otherwise, the controller fetches the required block from main memory.

Code Snippet

```
assign tag = address[8:7];
assign index = address[6:4];
assign offset = address[3:2];
assign hit =
(cache_valid[index] &&
 cache_tag[index]==tag);
```



6. Controller Design

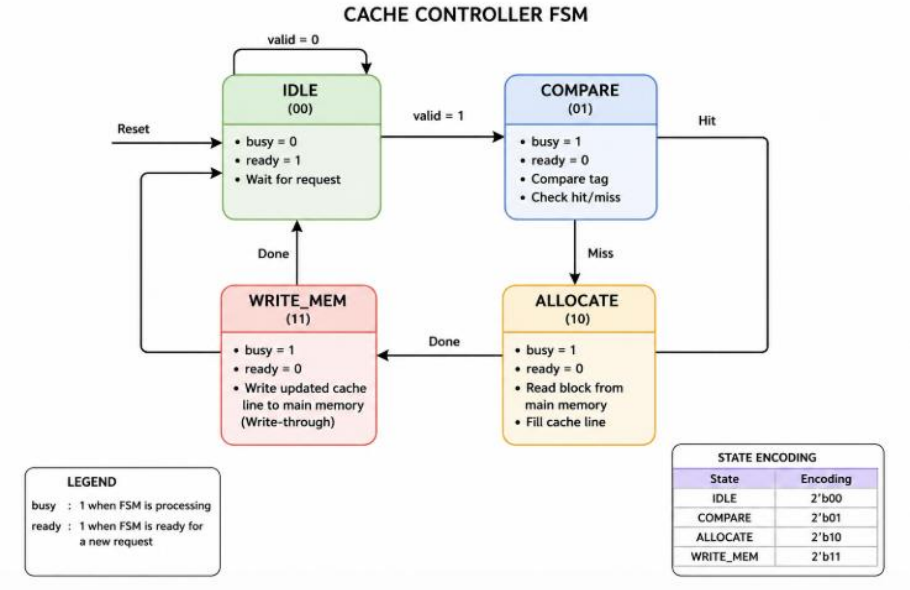
The controller is implemented using a Finite State Machine (FSM).

The FSM contains four states.

State	Function
IDLE	Waits for CPU request
COMPARE	Checks cache hit or miss
ALLOCATE	Reads block from main memory
WRITE_MEM	Writes updated block to memory

Code Snippet

```
localparam IDLE     = 2'b00;
localparam COMPARE   = 2'b01;
localparam ALLOCATE  = 2'b10;
localparam WRITE_MEM = 2'b11;
```



7. Sequence of Operations

Read Hit

1. CPU sends read request.
2. Controller enters COMPARE state.
3. Tag matches.
4. Hit signal becomes HIGH.
5. Requested word is returned.
6. Controller returns to IDLE.

Write Hit

1. CPU sends write request.
2. Tag comparison results in HIT.
3. Selected word inside cache line is updated.
4. Updated cache line is written to main memory.
5. Controller returns to IDLE.

Read Miss

1. CPU sends read request.
2. Tag comparison fails.
3. Controller enters ALLOCATE state.
4. Main memory sends the required block.
5. Cache line is updated.
6. Controller compares again.
7. Data is returned to CPU.

Write Miss

1. CPU sends write request.
2. Cache miss occurs.
3. Required block is fetched from main memory.
4. Block is stored in cache.
5. CPU data is written into the cache.
6. Updated cache line is written to main memory.

8. Generation of busy, ready and hit

hit

The hit signal is generated when

- Valid bit = 1
- Stored tag = Incoming tag

```
assign hit = (cache_valid[index] &&  cache_tag[index]==tag);
```

---

busy

The busy signal becomes HIGH whenever the controller is processing a request. It remains HIGH during the COMPARE, ALLOCATE and WRITE_MEM states.

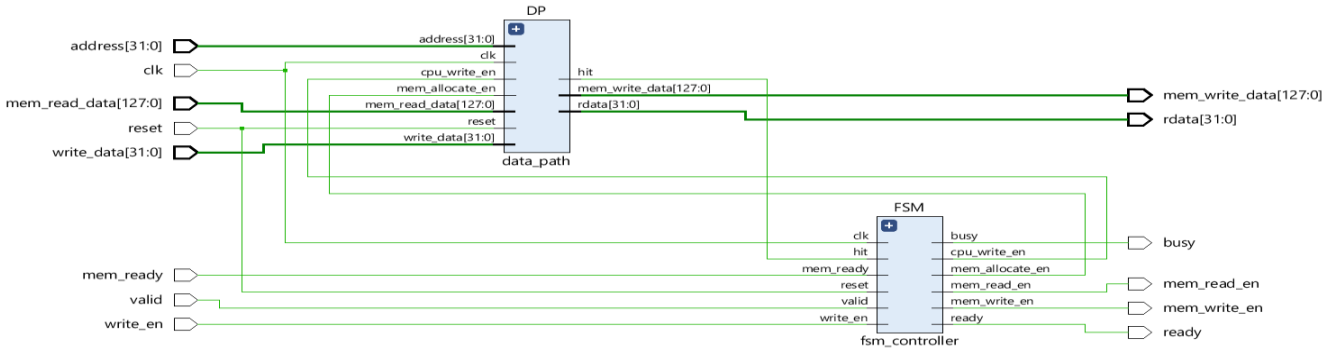
---

ready

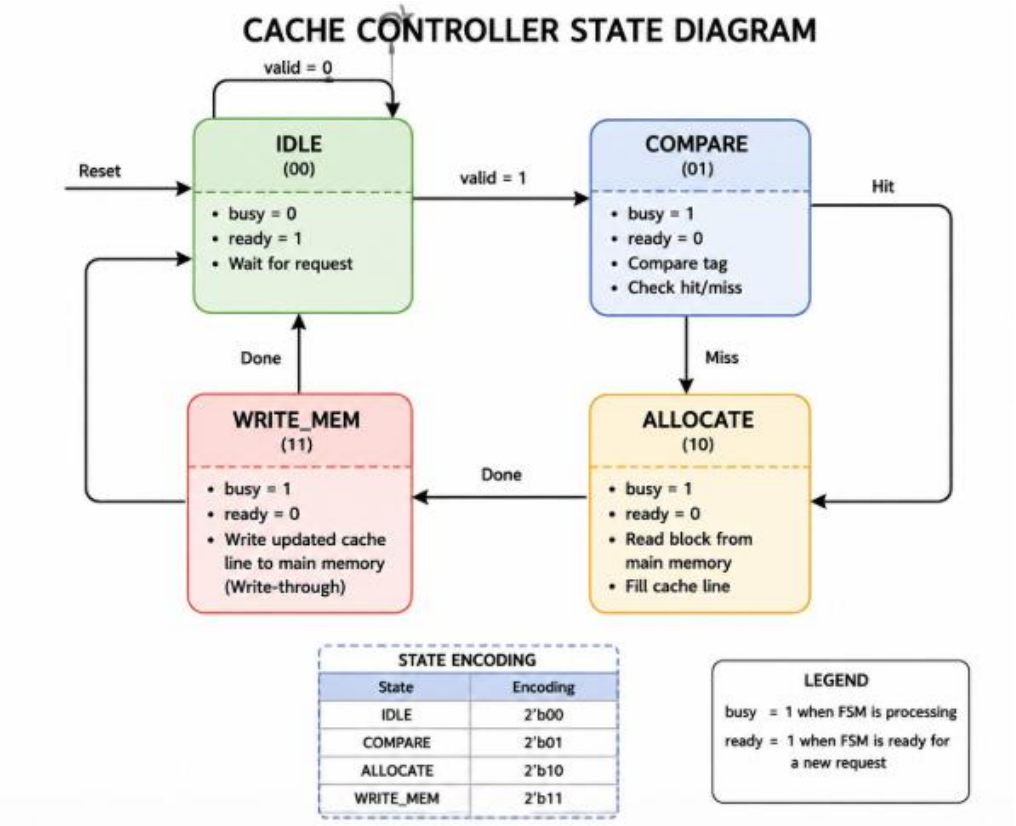
The ready signal becomes HIGH only when the controller is in the IDLE state and ready to accept a new CPU request.

---

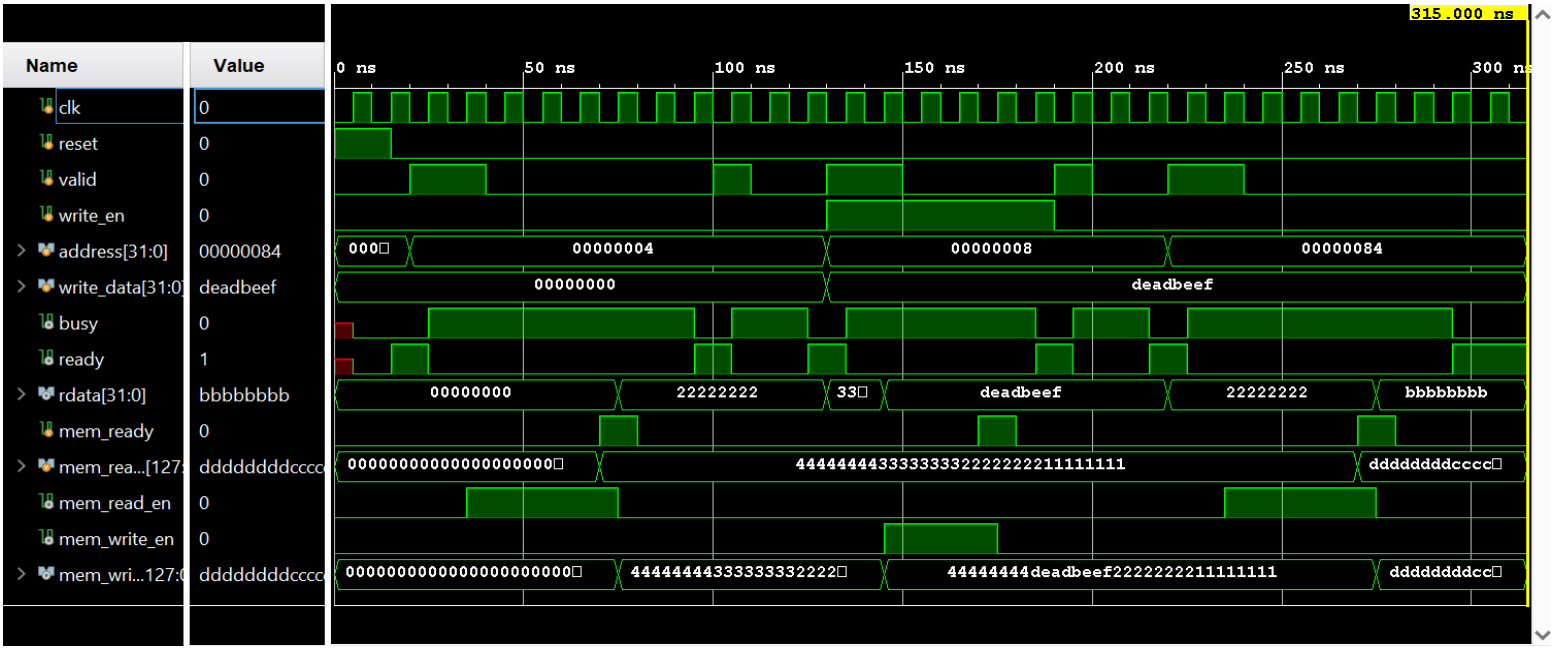
9. Block Diagram and State Diagram

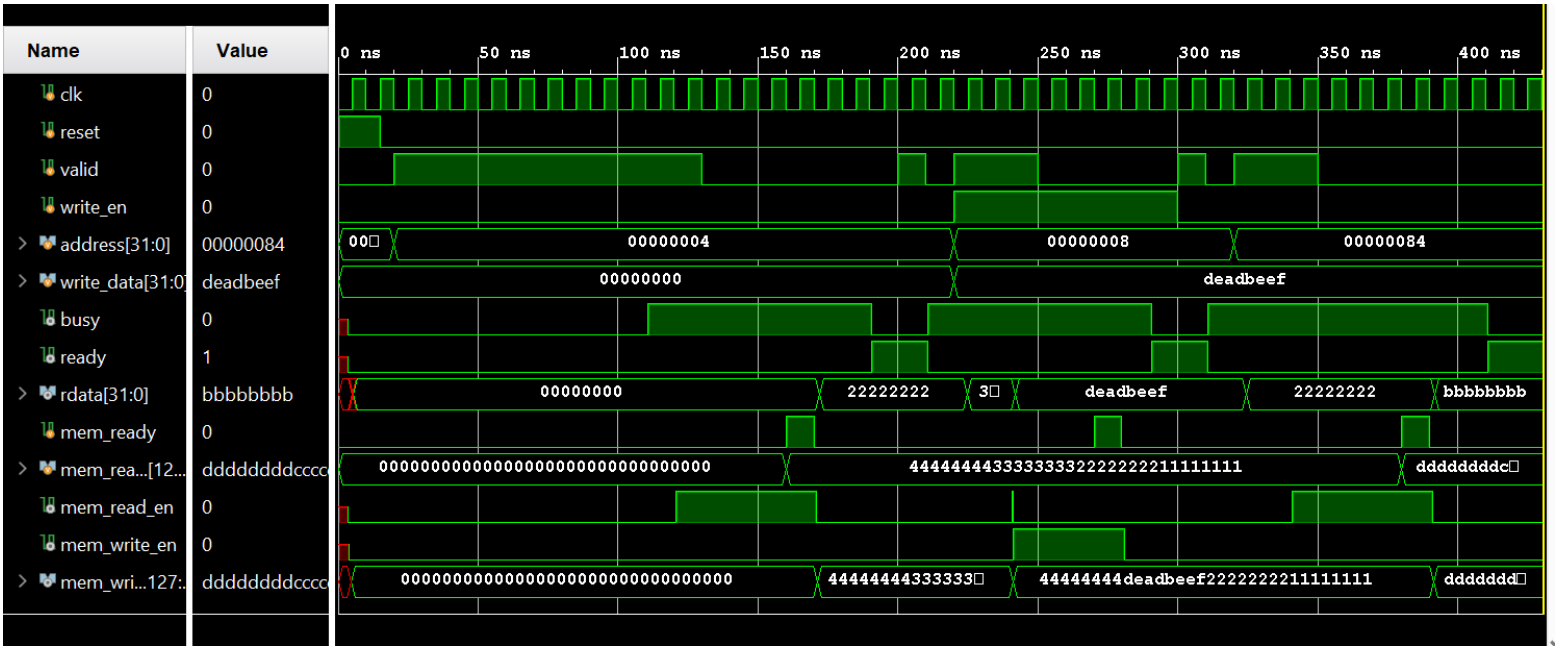


State Diagram



10. Behavioral Simulation and Post synthesis timing simulation





The cache controller was simulated using Xilinx Vivado. The behavioral simulation verifies the complete operation of the cache controller from reset until the final memory access.

The waveform demonstrates the following operations:

- **Cold Read Miss:** The first read request results in a cache miss. The controller asserts **mem_read_en**, fetches the required cache block from main memory, and allocates it into the cache.
- **Read Hit:** A second read to the same address results in a cache hit. The requested data is returned directly from the cache without accessing the main memory.
- **Write Hit:** The cache line is updated with the value **0xDEADBEEF**. Since a write-through policy is implemented, the updated cache line is also written to the main memory by asserting **mem_write_en**.
- **Read After Write:** Reading the same address again returns the updated value **0xDEADBEEF**, confirming that the cache contents were updated successfully.
- **Conflict Miss:** Accessing another address with the same cache index but a different tag results in a conflict miss. The controller fetches the new cache block from main memory and replaces the previous cache line.

The **busy** signal remains HIGH while the controller is processing a request, whereas the **ready** signal is asserted when the controller is ready to accept a new request. The **hit** signal correctly indicates whether the requested data is present in the cache.

The post-synthesis timing simulation produced the same functional behavior as the behavioral simulation, confirming that the synthesized hardware operates correctly.

11. Behavioral Simulation Analysis

The behavioral simulation verifies the correct operation of the cache controller under different memory access conditions. The waveform demonstrates the complete sequence of cache operations, including cache miss, cache hit, write-through operation, and cache line replacement.

Stage 1 – Cold Read Miss (0–100 ns)

Observed Signals

- Address = 0x00000004
- valid = 1
- busy = 1
- ready = 0
- mem_read_en = 1
- mem_ready = 1
- rdata = 0x22222222

Initially, the cache is empty after reset. When the processor requests address **0x00000004**, the controller detects a cache miss because the valid bit is cleared. The controller asserts **mem_read_en** to fetch the required 128-bit cache block from the main memory. After **mem_ready** becomes HIGH, the block **44444444_33333333_22222222_11111111** is loaded into the cache. Since the requested address has a word offset of **01**, the second word (**0x22222222**) is selected and returned through **rdata**.

Stage 2 – Read Hit (100–145 ns)

Observed Signals

- Address = 0x00000004
- valid = 1
- mem_read_en = 0
- rdata = 0x22222222

The same address is accessed again. Since the required cache block is already stored in the cache and the tag matches, a cache hit occurs. The controller returns the requested data directly from the cache without accessing the main memory. Therefore, **mem_read_en** remains LOW.

Stage 3 – Write Hit (145–225 ns)

Observed Signals

- Address = 0x00000008
- write_en = 1
- write_data = 0xDEADBEEF
- mem_write_en = 1
- mem_write_data = 44444444_DEADBEEF_22222222_11111111

The processor performs a write operation to address **0x00000008**. The address belongs to the same cache block, resulting in a write hit. The selected word within the cache line is updated with **0xDEADBEEF**. Since the controller implements a **write-through** policy, the updated 128-bit cache line is simultaneously written to the main memory through **mem_write_data**.

Stage 4 – Read After Write (225–255 ns)

Observed Signals

- Address = 0x00000008
- rdata = 0xDEADBEEF

The same address is read immediately after the write operation. The cache returns **0xDEADBEEF**, confirming that the previous write operation successfully updated the cache contents.

Stage 5 – Conflict Miss (255–315 ns)

Observed Signals

- Address = 0x00000084
- mem_read_en = 1
- mem_ready = 1
- rdata = 0xBBBBBBBB

Finally, address **0x00000084** is accessed. Although it maps to the **same cache index** as the previous block, its tag is different. The controller detects a conflict miss and requests the new block from the main memory. The existing cache line is replaced with the new block **DDDDDDDD_CCCCCCCC_BBBBBBBB_AAAAAAAA**. Based on the word offset, the controller returns **0xBBBBBBBB** through **rdata**, confirming successful cache replacement.